

Ứng dụng và cài đặt đơn giản của quy hoạch tuyến tính

Li Yuqian

2007

Nguồn gốc: Simple applications and implementation of linear programming

IOI China candidate team papers / 2007 / day 1 / Li Yuqian

Tóm tắt

Quy hoạch tuyến tính được dùng rất rộng rãi trong đời sống thực tế và đã tạo ra rất nhiều giá trị. Tuy vậy, trong các cuộc thi tin học, ứng dụng của nó vẫn còn hiếm. Bài viết này giới thiệu sơ lược một số ứng dụng của quy hoạch tuyến tính và cách cài đặt thuật toán để giải nó, với hy vọng quy hoạch tuyến tính có thể tỏa sáng trong thi tin học như luồng mạng.

Bài viết gồm ba phần. Phần thứ nhất giới thiệu và định nghĩa quy hoạch tuyến tính. Phần thứ hai trình bày vài ứng dụng đơn giản, trong đó có một ứng dụng kinh điển là luồng đa hàng hóa. Phần thứ ba trình bày cách dùng thuật toán đơn hình (simplex) để giải quy hoạch tuyến tính.

Với đa số thí sinh, tự viết một chương trình giải quy hoạch tuyến tính thường đáng sợ hơn việc dựng mô hình, dù độ khó chưa chắc lớn hơn. Hơn nữa, thuật toán đơn hình không có bảo đảm đa thức, nên nếu không thực nghiệm thì khó biết tốc độ thực tế của nó. Vì vậy bài viết tập trung vào phần thứ ba: mô tả khá chi tiết các điểm cài đặt, nêu chứng minh ngắn gọn, thử nghiệm tốc độ và so sánh với các phần mềm toán học MATLAB, LINDO. Kết quả cho thấy tốc độ thực tế của đơn hình rất tốt. Một chương trình khoảng 200 dòng thường đủ để giải bài tổng quát; trong một số trường hợp, khoảng 100 dòng cũng đủ.

Từ khóa. Quy hoạch tuyến tính; thuật toán đơn hình; luồng đa hàng hóa.

Dẫn nhập

Khi các thuật toán mạnh được phát triển và áp dụng, quy hoạch tuyến tính giải được ngày càng nhiều bài toán. Trong lịch sử, hiếm có phương pháp toán học nào trực tiếp tạo ra lượng của cải lớn và ảnh hưởng rõ rệt đến tiến trình lịch sử như quy hoạch tuyến tính.

Một ví dụ thường được nhắc đến là trong Chiến tranh vùng Vịnh, quân đội Hoa Kỳ đã dùng quy hoạch tuyến tính để giải hiệu quả bài toán điều phối hậu cần và vũ khí, góp phần quan trọng vào thắng lợi. Nếu Chiến tranh thế giới thứ nhất có thể được gọi là “cuộc chiến hóa học” vì thuốc nổ, Chiến tranh thế giới thứ hai là “cuộc chiến vật lý” vì bom nguyên tử, thì Chiến tranh vùng Vịnh từng được gọi là “cuộc chiến toán học” vì quy hoạch tuyến tính.

Sức mạnh thực tế của quy hoạch tuyến tính là điều không cần nghi ngờ. Trong thi tin học, ánh sáng của nó chưa xuất hiện nhiều trước mắt chúng ta; qua việc học và hiểu nó, ta có thể dần cảm nhận được giá trị ấy.

1 Giới thiệu và định nghĩa

Ta thường gặp những bài toán cần cực đại hóa hoặc cực tiểu hóa một mục tiêu. Chúng cũng thường bị giới hạn bởi tài nguyên hoặc điều kiện khác, hoặc buộc phải đánh đổi giữa các lựa chọn không thể đồng thời đạt được. Nếu mục tiêu có thể biểu diễn bằng một hàm tuyến tính, còn các giới hạn hoặc đánh đổi có thể biểu diễn bằng các đẳng thức hoặc bất đẳng thức tuyến tính, ta có thể mô tả bài toán đó như một bài toán quy hoạch tuyến tính.

Một ví dụ tranh cử

Giả sử bạn tranh cử thị trưởng. Để đắc cử, bạn cần 50 nghìn phiếu của cư dân thành thị, 100 nghìn phiếu của cư dân ngoại ô và 25 nghìn phiếu của cư dân nông thôn. Bạn có bốn chính sách để tăng phiếu:

1. xây dựng đường sá;
2. tăng kiểm soát súng;
3. phát trợ cấp nông nghiệp;
4. giảm thuế xăng dầu.

Sau khi đánh giá, ta có bảng sau. Mỗi ô cho biết nếu chi thêm 10 nghìn đồng vào chính sách tương ứng, số phiếu ở khu vực tương ứng thay đổi bao nhiêu nghìn phiếu.

Chính sách	Thành thị	Ngoại ô	Nông thôn
Xây đường	-2	5	3
Kiểm soát súng	8	2	-5
Trợ cấp nông nghiệp	0	0	10
Giảm thuế xăng dầu	10	0	-2

Ví dụ, ô đầu tiên -2 nghĩa là mỗi 10 nghìn đồng chi cho xây đường làm giảm 2 nghìn phiếu thành thị; ô hàng đầu cột thứ hai nghĩa là tăng 5 nghìn phiếu ngoại ô; ô ở hàng kiểm soát súng cột nông thôn nghĩa là giảm 5 nghìn phiếu nông thôn. Giả sử ban đầu bạn có 0 phiếu ở cả ba khu vực. Hãy dùng ít chi phí nhất để đạt đủ số phiếu.

Đặt x_1, x_2, x_3, x_4 là số đơn vị chi cho bốn chính sách. Bài toán trở thành:

$$\text{cực tiểu } x_1 + x_2 + x_3 + x_4$$

với các ràng buộc

$$\begin{aligned} -2x_1 + 8x_2 + 0x_3 + 10x_4 &\geq 50 && (\text{thành thị}), \\ 5x_1 + 2x_2 + 0x_3 + 0x_4 &\geq 100 && (\text{ngoại ô}), \\ 3x_1 - 5x_2 + 10x_3 - 2x_4 &\geq 25 && (\text{nông thôn}), \\ x_1, x_2, x_3, x_4 &\geq 0 && (\text{không thể chi âm}). \end{aligned}$$

Định nghĩa

Quy hoạch tuyến tính là bài toán tối ưu hóa một hàm tuyến tính dưới một số điều kiện là đẳng thức hoặc bất đẳng thức tuyến tính.

Với các số thực $a_1, a_2, \dots, a_n$ và các biến $x_1, x_2, \dots, x_n$, một hàm tuyến tính có dạng

$$f(x_1, x_2, \dots, x_n) = a_1x_1 + a_2x_2 + \dots + a_nx_n = \sum_{j=1}^n a_jx_j.$$

Nếu f là hàm tuyến tính thì $f(x_1, \dots, x_n) = b$ là đẳng thức tuyến tính, còn $f(x_1, \dots, x_n) \leq b$ và $f(x_1, \dots, x_n) \geq b$ là bất đẳng thức tuyến tính. Những điều kiện này được gọi chung là ràng buộc tuyến tính.

Một **nghiệm** của quy hoạch tuyến tính là một vector $(y_1, y_2, \dots, y_n)$ sao cho khi đặt $x_i = y_i$, hàm mục tiêu đạt tối ưu và các điều kiện đều được thỏa. Một **nghiệm khả thi** chỉ cần thỏa điều kiện, chưa chắc tối ưu. **Giá trị tối ưu** là giá trị tốt nhất của hàm mục tiêu trong miền khả thi.

Thông thường ta có thể viết một bài toán quy hoạch tuyến tính dưới dạng:

$$\begin{array}{ll} \text{cực tiểu hoặc cực đại} & f(x_1, x_2, \dots, x_n) \\ \text{thỏa} & \begin{aligned} p_i(x_1, x_2, \dots, x_n) &\leq a_i, \\ q_i(x_1, x_2, \dots, x_n) &= b_i, \\ r_i(x_1, x_2, \dots, x_n) &\geq c_i. \end{aligned} \end{array}$$

Ở đây f là hàm mục tiêu; p_i, q_i, r_i lần lượt đại diện cho các ràng buộc dạng không vượt quá, bằng, và không nhỏ hơn.

Ví dụ, cả hai bài toán sau đều là quy hoạch tuyến tính:

$$\begin{array}{ll} \text{cực đại} & 2x_1 - 3x_2 + 3x_3 \\ \text{thỏa} & \begin{aligned} x_1 + x_2 - x_3 &= 7, \\ x_1 - 2x_2 + 2x_3 &\leq 4, \end{aligned} \end{array}$$

và

$$\begin{array}{ll} \text{cực tiểu} & 2x_1 + 7x_2 \\ \text{thỏa} & \begin{aligned} x_1 &= 7, \\ 3x_1 + x_2 &\geq 24.5, \\ x_2 &\geq 3, \\ x_3 &\leq -1. \end{aligned} \end{array}$$

Cần chú ý rằng hệ số trong quy hoạch tuyến tính không cần là số nguyên hay số hữu tỉ; chúng có thể là số thực bất kỳ. Nghiệm tối ưu cũng không bắt buộc là số nguyên hoặc hữu tỉ. Nếu ta thêm điều kiện nghiệm phải nguyên, bài toán trở thành quy hoạch nguyên, thuộc lớp khó hơn nhiều; hiện chưa có thuật toán đa thức tổng quát cho nó.

2 Một số ứng dụng đơn giản

Phần này trình bày ba ứng dụng. Hai ứng dụng đầu là phân bổ tài nguyên và cung ứng vật tư tối ưu, tương đối đơn giản. Ứng dụng cuối là luồng đa hàng hóa, phức tạp hơn.

2.1 Phân bổ tài nguyên

Có m loại tài nguyên và n dự án. Mỗi tài nguyên hữu hạn, giới hạn của tài nguyên thứ j là b_j với $1 \leq j \leq m$. Nếu dự án thứ i tạo ra lượng thành quả x_i , ta thu lợi $c_i x_i$, đồng thời tiêu thụ $a_{ij} x_i$ đơn vị tài nguyên loại j . Cần tối đa hóa tổng lợi ích.

Ví dụ, một nhà máy có lần lượt b_1, b_2, b_3 tấn thép, gỗ, nhựa. Nhà máy có thể sản xuất 4 loại sản phẩm. Mỗi tấn sản phẩm loại i đem lại c_i vạn đồng lợi nhuận, nhưng tiêu thụ a_{i1} tấn thép, a_{i2} tấn gỗ và

a_{i3} tấn nhựa. Đặt sản lượng bốn loại là x_1, x_2, x_3, x_4 , ta có:

$$\begin{array}{ll} \text{cực đại} & c_1x_1 + c_2x_2 + c_3x_3 + c_4x_4 \\ \text{thỏa} & a_{11}x_1 + a_{21}x_2 + a_{31}x_3 + a_{41}x_4 \leq b_1, \\ & a_{12}x_1 + a_{22}x_2 + a_{32}x_3 + a_{42}x_4 \leq b_2, \\ & a_{13}x_1 + a_{23}x_2 + a_{33}x_3 + a_{43}x_4 \leq b_3, \\ & x_1, x_2, x_3, x_4 \geq 0. \end{array}$$

Viết gọn:

$$\begin{array}{ll} \text{cực đại} & \sum_{i=1}^4 c_i x_i \\ \text{thỏa} & \sum_{i=1}^4 a_{ij} x_i \leq b_j \quad (1 \leq j \leq 3), \\ & x_i \geq 0 \quad (1 \leq i \leq 4). \end{array}$$

Dạng tổng quát là

$$\begin{array}{ll} \text{cực đại} & \sum_{i=1}^n c_i x_i \\ \text{thỏa} & \sum_{i=1}^n a_{ij} x_i \leq b_j \quad (1 \leq j \leq m), \\ & x_i \geq 0 \quad (1 \leq i \leq n). \end{array}$$

2.2 Cung ứng vật tư tối ưu

Có m loại nhu cầu cần được đáp ứng; nhu cầu thứ j cần ít nhất b_j đơn vị. Có n loại vật tư, mỗi đơn vị vật tư loại i cung cấp a_{ij} đơn vị cho nhu cầu thứ j . Nếu cung cấp x_i đơn vị vật tư loại i , chi phí là $c_i x_i$. Cần cực tiểu hóa tổng chi phí trong khi vẫn đáp ứng mọi nhu cầu.

Ví dụ, một người mỗi ngày cần ít nhất b_1 gam đường, b_2 gam chất béo, b_3 gam protein và b_4 gam vitamin. Cơm có giá c_1 đồng mỗi gam, mỗi gam cung cấp $a_{11}, a_{12}, a_{13}, a_{14}$ gam tương ứng cho bốn nhóm dinh dưỡng; rau và thịt được mô tả tương tự. Nếu mỗi ngày ăn x_1 gam cơm, x_2 gam rau và x_3 gam thịt, bài toán là

$$\begin{array}{ll} \text{cực tiểu} & c_1x_1 + c_2x_2 + c_3x_3 \\ \text{thỏa} & a_{11}x_1 + a_{21}x_2 + a_{31}x_3 \geq b_1, \\ & a_{12}x_1 + a_{22}x_2 + a_{32}x_3 \geq b_2, \\ & a_{13}x_1 + a_{23}x_2 + a_{33}x_3 \geq b_3, \\ & a_{14}x_1 + a_{24}x_2 + a_{34}x_3 \geq b_4, \\ & x_1, x_2, x_3 \geq 0. \end{array}$$

Dạng tổng quát là

$$\begin{array}{ll} \text{cực tiểu} & \sum_{i=1}^n c_i x_i \\ \text{thỏa} & \sum_{i=1}^n a_{ij} x_i \geq b_j \quad (1 \leq j \leq m), \\ & x_i \geq 0 \quad (1 \leq i \leq n). \end{array}$$

2.3 Luồng đa hàng hóa

Nhìn tiêu đề này, nhiều người sẽ liên tưởng đến luồng mạng thông thường: có cách giải đơn giản, hiệu quả và ứng dụng rộng. Nhiều mô hình luồng mạng kinh điển tinh tế đến mức đáng kinh ngạc. Luồng đa hàng hóa có nhiều điểm giống luồng mạng, nhưng đến nay cách giải hiệu quả tổng quát của nó vẫn là quy

hoạch tuyến tính. Tác giả tin rằng phạm vi ứng dụng của nó không hề kém luồng mạng, thậm chí mạnh hơn, vì nó không chỉ bao gồm bài toán một nguồn một đích mà còn xử lý được nhiều cặp nguồn, đích.

Về cơ bản, luồng đa hàng hóa giống luồng mạng thông thường, khác ở chỗ có k cặp nguồn và đích. Gọi nguồn của hàng hóa thứ i là s_i , đích là t_i với $1 \leq i \leq k$. Bài toán yêu cầu một luồng khả thi sao cho lượng từ s_i đến t_i là d_i .

Cụ thể, cho đồ thị có hướng gồm V đỉnh và E cạnh. Cạnh thứ e đi từ u_e đến v_e và có dung lượng không âm c_e . Có k hàng hóa, hàng hóa thứ i được biểu diễn bởi (s_i, t_i, d_i) . Ta định nghĩa hàm $f_i(u_e, v_e)$ là lượng hàng hóa i đi trên cạnh e . Hàm này thỏa bảo toàn luồng: ngoài s_i và t_i , tổng lượng vào một đỉnh bằng tổng lượng ra. Tổng lượng của mọi hàng hóa trên một cạnh không được vượt dung lượng cạnh đó. Cần tìm các hàm f_i thỏa điều kiện.

Ví dụ, một công ty cần dùng đường sắt vận chuyển k loại hàng hóa, lần lượt từ thành phố s_i đến t_i , mỗi ngày cần chuyển d_i đơn vị. Biết giới hạn vận chuyển hằng ngày của các tuyến đường sắt giữa các thành phố. Giả sử hàng hóa có thể chia tùy ý thành nhiều phần và đi theo nhiều tuyến khác nhau. Hãy tìm một phương án vận chuyển khả thi.

Mô hình quy hoạch tuyến tính cho luồng đa hàng hóa là:

$$\begin{aligned} \text{cực tiểu} \quad & 0 \\ \text{thỏa} \quad & \sum_{i=1}^k f_i(u_e, v_e) \leq c_e \quad (1 \leq e \leq E), \\ & \sum_{p: v_p=v} f_i(u_p, v_p) = \sum_{q: u_q=v} f_i(u_q, v_q) \\ & \quad (1 \leq i \leq k, 1 \leq v \leq V, v \neq s_i, t_i), \\ & \sum_{e: u_e=s_i} f_i(u_e, v_e) = d_i \quad (1 \leq i \leq k), \\ & f_i(u_e, v_e) \geq 0 \quad (1 \leq i \leq k, 1 \leq e \leq E). \end{aligned}$$

Ở đây cực tiểu hóa 0 nghĩa là chỉ cần tìm nghiệm khả thi; mọi hệ số trong hàm mục tiêu đều bằng 0. Cách viết này khác một chút so với trong *Introduction to Algorithms*, chủ yếu để tiện xử lý trường hợp có cạnh song song.

Nếu mỗi cạnh còn có chi phí đơn vị Cost_e , chi phí trên cạnh bằng tổng luồng đi qua cạnh nhân với Cost_e . Khi đó ta không chỉ cần luồng khả thi, mà còn cần luồng khả thi có chi phí nhỏ nhất. Điều kiện giữ nguyên, còn hàm mục tiêu của luồng đa hàng hóa chi phí nhỏ nhất là

$$\text{cực tiểu} \quad \sum_{e=1}^E \text{Cost}_e \sum_{i=1}^k f_i(u_e, v_e).$$

Ví dụ chi phí nhỏ nhất

Xét một bài sửa từ NWERC 2006 D. Trên một bản đồ, một công ty đường sắt có 4 dự án. Dự án thứ i cần xây một tuyến từ thành phố s_i đến t_i . Có một số đoạn đường sắt có thể chọn xây, mỗi đoạn đi từ một thành phố đến một thành phố khác và có chi phí xây dựng đã biết. Các dự án độc lập, nên mỗi đoạn đường sắt chỉ có thể thuộc về một dự án, được dự án ấy xây và sau đó chỉ dự án ấy sử dụng. Hãy tìm chi phí nhỏ nhất để hoàn thành cả 4 dự án.

Ta dựng đồ thị với thành phố là đỉnh, đoạn đường sắt là cạnh. Mỗi cạnh có dung lượng 1, chi phí đơn vị bằng chi phí xây đoạn đó, $k = 4$, các s_i, t_i giữ nguyên và mọi $d_i = 1$.

Nhìn kỹ mô hình này, có một vấn đề: trong bài gốc, một hàng hóa trên một cạnh phải là 0 hoặc 1, không được là số thực khác; trong khi luồng đa hàng hóa cho phép lượng trên cạnh là số thực bất kỳ. Liệu đáp án thu được có sai không? May mắn là không, vì ta chỉ cần chi phí nhỏ nhất chứ không nhất thiết cần xuất ngay phương án nguyên. Giá trị hàm mục tiêu thu được bằng chi phí nhỏ nhất, dù một số $f_i(u_e, v_e)$ trong nghiệm tuyến tính có thể không phải 0 hoặc 1.

Lý do là mô hình tuyến tính được nói lỏng so với bài gốc, nên giá trị tối ưu của nó không lớn hơn chi phí tối ưu của bài gốc. Nếu ta chứng minh được với giá trị tối ưu ấy luôn có thể dựng một phương án thỏa bài gốc, thì giá trị đó chính là đáp án. Điểm này phụ thuộc vào cách cài đặt đơn hình ở phần sau: với mô hình này, thuật toán đơn hình trả về một nghiệm nguyên. Do ràng buộc dung lượng cạnh là 1, lượng một hàng hóa trên một cạnh trong nghiệm ấy chỉ có thể là 0 hoặc 1. Vì vậy nếu dùng đơn hình để giải, ta còn có thể nhận được một phương án cụ thể.

3 Cài đặt thuật toán đơn hình

Phần này gồm sáu mục. Năm mục đầu trình bày thuật toán đơn hình: dạng chuẩn và dạng lỏng, phép pivot, chương trình chính, khởi tạo, một số chi tiết cài đặt và tối ưu. Mục cuối ghi lại thực nghiệm so sánh chương trình của tác giả với LINDO và MATLAB.

3.1 Dạng chuẩn và dạng lỏng

Để thuận tiện, ta muốn đưa quy hoạch tuyến tính về một dạng thống nhất. Khi biến đổi từ dạng này sang dạng khác, trước hết cần bảo đảm tính tương đương. Nếu hai bài toán đều cực đại hóa hoặc đều cực tiểu hóa, chúng tương đương khi mỗi nghiệm khả thi của bài này có một nghiệm khả thi của bài kia với cùng giá trị mục tiêu, và ngược lại. Khi đó giá trị tối ưu bằng nhau. Nếu một bài cực đại hóa còn bài kia cực tiểu hóa, chúng tương đương khi giá trị mục tiêu tương ứng trái dấu; khi đó giá trị tối ưu cũng trái dấu.

Có thể hiểu đơn giản rằng nếu B thu được từ A bằng các biến đổi chấp nhận được thì A và B tương đương. Các biến đổi đó gồm nhân hai vế bất đẳng thức với -1 , chuyển vế, thay một đẳng thức bằng hai bất đẳng thức, nhân hàm mục tiêu với -1 và đổi cực đại thành cực tiểu hoặc ngược lại.

Dạng chuẩn trong bài viết này là: hàm mục tiêu cần cực đại hóa, mọi ràng buộc tuyến tính là bất đẳng thức dạng không vượt quá, và mọi biến đều có ràng buộc không âm. Ta viết:

$$\begin{array}{ll} \text{cực đại} & f(x_1, x_2, \dots, x_n) \\ \text{thỏa} & x_i \geq 0 \quad (1 \leq i \leq n), \\ & p_i(x_1, x_2, \dots, x_n) \leq a_i \quad (1 \leq i \leq m). \end{array}$$

Một bài toán không ở dạng chuẩn có thể vi phạm bốn điểm sau:

1. hàm mục tiêu là cực tiểu;
2. có ràng buộc đẳng thức;
3. có bất đẳng thức dạng không nhỏ hơn;
4. có biến không bị ràng buộc không âm.

Ta xử lý từng trường hợp:

1. Nếu A cực tiểu hóa $\sum_{i=1}^n c_i x_i$, lập bài toán B với cùng ràng buộc nhưng cực đại hóa $\sum_{i=1}^n (-c_i) x_i$.
2. Nếu có điều kiện $q_i(x_1, \dots, x_n) = b_i$, thay nó bằng hai điều kiện

$$q_i(x_1, \dots, x_n) \leq b_i \quad \text{và} \quad q_i(x_1, \dots, x_n) \geq b_i.$$

3. Nếu có điều kiện $\sum_{i=1}^n a_i x_i \geq b$, thay bằng $\sum_{i=1}^n (-a_i) x_i \leq -b$.
4. Nếu biến x_i không có ràng buộc không âm, thay mọi x_i bằng hiệu của hai biến mới $x_{i1} - x_{i2}$ và thêm $x_{i1} \geq 0, x_{i2} \geq 0$. Với một nghiệm của bài gốc, nếu $x_i \geq 0$ thì lấy $x_{i1} = x_i, x_{i2} = 0$; nếu $x_i < 0$ thì lấy $x_{i1} = 0, x_{i2} = -x_i$. Chiều ngược lại là hiển nhiên.

Dạng chuẩn có thể được biểu diễn bằng một vector c , một vector b và một ma trận A . Vector c có n phần tử, biểu diễn hàm mục tiêu $\sum_{i=1}^n c_i x_i$. Vector b có m phần tử. Ma trận A cùng b biểu diễn m bất đẳng thức:

$$\sum_{j=1}^n a_{ij} x_j \leq b_i \quad (1 \leq i \leq m).$$

Trong thuật toán đơn hình, ta thường đổi dạng chuẩn thành **dạng lỏng**. Ta thêm m biến mới $x_{n+1}, \dots, x_{n+m}$, trong đó

$$x_{n+i} = b_i - \sum_{j=1}^n a_{ij} x_j.$$

Khi đó bài toán trở thành:

$$\begin{aligned} \text{cực đại} \quad & f(x_1, x_2, \dots, x_n) \\ \text{thỏa} \quad & x_{n+i} = b_i - \sum_{j=1}^n a_{ij} x_j \quad (1 \leq i \leq m), \\ & x_i \geq 0 \quad (1 \leq i \leq n+m). \end{aligned}$$

Một dạng lỏng được biểu diễn bằng bộ sáu (N, B, A, b, c, v) . Ở đây B là tập m chỉ số nguyên, N là tập n chỉ số nguyên, A là ma trận thực kích thước $(n+m) \times (n+m)$, b, c là hai vector thực kích thước $n+m$, và v là một số thực. Nó biểu diễn:

$$\begin{aligned} \text{cực đại} \quad & v + \sum_{i \in N} c_i x_i \\ \text{thỏa} \quad & x_i = b_i - \sum_{j \in N} A_{ij} x_j \quad (i \in B), \\ & x_i \geq 0 \quad (1 \leq i \leq n+m). \end{aligned}$$

Khi chuyển từ dạng chuẩn ban đầu, ta có $v = 0$, $N = \{1, \dots, n\}$, $B = \{n+1, \dots, n+m\}$ và $A_{n+i,j} = a_{ij}$. Các biến có chỉ số trong N gọi là biến phi cơ sở (nonbasic), còn các biến có chỉ số trong B gọi là biến cơ sở (basic). Trong quá trình thuật toán, một phần tử của B sẽ đổi chỗ với một phần tử của N ; đồng thời A, b, c, v được cập nhật để dạng mới tương đương dạng cũ. Đó là phép **pivot**.

Nếu mọi phần tử của b không âm, dạng lỏng có ngay một nghiệm khả thi: đặt mọi biến phi cơ sở bằng 0, khi đó biến cơ sở $x_i = b_i$ và giá trị mục tiêu là v . Bằng một số hữu hạn phép pivot, ta sẽ thu được dạng lỏng mà nghiệm khả thi này là nghiệm tối ưu, và v là giá trị tối ưu.

3.2 Phép pivot

Phép pivot nhận hai tham số l và e , nghĩa là biến cơ sở x_l đổi chỗ với biến phi cơ sở x_e .

Trong dạng lỏng hiện tại có phương trình

$$x_l = b_l - \sum_{j \in N} A_{lj} x_j.$$

Sau khi đổi vai trò của x_l và x_e , ta viết lại:

$$A_{le} x_e = b_l - \sum_{\substack{j \in N \\ j \neq e}} A_{lj} x_j - x_l,$$

tức

$$x_e = \frac{b_l}{A_{le}} - \sum_{\substack{j \in N \\ j \neq e}} \frac{A_{lj}}{A_{le}} x_j - \frac{x_l}{A_{le}}.$$

Thay phương trình này vào hàm mục tiêu và các phương trình còn lại, ta đạt được việc đổi chỗ.

Mã giả:

```

pivot(l, e)
    tb[e] = b[l] / A[l][e]
    tA[e][l] = 1 / A[l][e]
    for i in N, i != e
        tA[e][i] = A[l][i] / A[l][e]
    // đã có phương trình với xe ở về trái

    for i in B, i != l
        tb[i] = b[i] - A[i][e] * tb[e]
        for j in N, j != e
            tA[i][j] = A[i][j] - A[i][e] * tA[e][j]
    // thay vào các phương trình khác

    tv = v + tb[e] * c[e]
    tc[l] = -tA[e][l] * c[e]
    for i in N, i != e
        tc[i] = c[i] - c[e] * tA[e][i]
    // thay vào hàm mục tiêu

    v = tv
    N = N - {e} + {l}
    B = B - {l} + {e}
    A = tA
    b = tb
    c = tc

```

3.3 Chương trình chính

Chương trình đơn hình gồm hai phần: khởi tạo và tối ưu hóa. Khởi tạo biến một dạng lỏng bất kỳ thành dạng lỏng có mọi $b_i \geq 0$, hoặc kết luận bài toán vô nghiệm. Tối ưu hóa nhận một dạng lỏng có mọi $b_i \geq 0$, rồi tìm dạng mà v là giá trị tối ưu, hoặc kết luận giá trị tối ưu là vô hạn.

```

simplex
    init
    opt

```

Thủ tục `opt` liên tục chọn một biến vào e và một biến ra l , rồi thực hiện `pivot`, cho đến khi biết bài toán không bị chặn hoặc đã tìm được nghiệm.

```

opt
    while true
        if c[i] <= 0 for every i in N
            return solution found
        else choose e with c[e] > 0

        delta = infinity
        for i in B
            if A[i][e] > 0 and delta > b[i] / A[i][e]
                delta = b[i] / A[i][e]

```



```

l = i

if delta == infinity
    return unbounded
else
    pivot(l, e)

```

Trong opt, ban đầu mọi b_i đều không âm. Cách chọn l bảo đảm sau mỗi pivot, mọi b_i vẫn không âm: ta chọn b_l/A_{le} nhỏ nhất trong các hàng có $A_{le} > 0$. Chỉ cần nhìn lại công thức cập nhật b trong pivot là thấy điều này.

Vì ở mọi bước b không âm, đặt mọi biến phi cơ sở bằng 0 luôn cho một nghiệm khả thi, và khi đó giá trị mục tiêu là v . Trong opt, v không giảm, vì ta chọn e sao cho $c_e > 0$, còn x_e ban đầu bằng 0 và chỉ tăng. Trong đa số pivot, v tăng; đôi khi nó có thể không đổi. Ý tưởng là tăng x_e nhiều nhất có thể trong khi không làm b âm. Chính điểm này giải thích vì sao đơn hình có thể xử lý ví dụ luồng đa hàng hóa chi phí nhỏ nhất: giống như thuật toán tăng luồng, với dung lượng nguyên, lượng tăng trên cạnh dung lượng nguyên không sinh ra giá trị phân số.

Liệu v có thể mãi không đổi và khiến chương trình lặp vô hạn không? Xác suất gặp tình huống ấy rất nhỏ, và có thể tránh bằng một quy tắc đơn giản: khi chọn e và l , ưu tiên thứ tự từ điển, tức chỉ số nhỏ hơn được ưu tiên. Đây là quy tắc Bland.

Ta kết luận không bị chặn khi tồn tại biến phi cơ sở x_i với $c_i > 0$ và với mọi $j \in B$ đều có $A_{ji} \leq 0$. Khi đó đặt mọi biến phi cơ sở khác bằng 0 và cho x_i tiến tới vô hạn; các biến cơ sở $x_j = b_j - A_{ji}x_i$ vẫn không âm, còn hàm mục tiêu tiến tới vô hạn.

Khi mọi $i \in N$ đều có $c_i \leq 0$, vì sao v là tối ưu? Một cách hiểu trực quan là: khi các biến phi cơ sở được cố định, biến cơ sở cũng được xác định, do đó một nghiệm khả thi có thể xem như được quyết định bởi các biến phi cơ sở hiện tại. Các biến đều không âm, nên từ trạng thái các biến phi cơ sở bằng 0, chúng chỉ có thể tăng. Nhưng hệ số của chúng trong hàm mục tiêu đều không dương, nên tăng chúng không thể làm mục tiêu lớn hơn. Vì vậy v là giá trị tối ưu.

Về độ phức tạp, quy tắc Bland tránh lặp vô hạn, nên một dạng lỏng xuất hiện nhiều nhất một lần. Một dạng lỏng được xác định bởi tập N ; khi N đã biết, B, A, b, c, v được xác định duy nhất. Vì vậy số dạng lỏng tối đa là $\binom{n+m}{n}$, và độ phức tạp bị chặn bởi số này nhân với chi phí một pivot. Đây chỉ là cận trên lý thuyết; tốc độ thực tế thường thấp hơn rất nhiều.

3.4 Khởi tạo

Nếu ban đầu mọi b_i đều không âm, ta có thể trả về ngay. Nếu không, để thu được một dạng lỏng ban đầu có mọi $b_i \geq 0$, ta dựng một bài toán phụ.

Giả sử điều kiện của dạng lỏng gốc là

$$x_i = b_i - \sum_{j \in N} A_{ij}x_j \quad (i \in B).$$

Trong bài toán phụ, điều kiện trở thành

$$x_i = b_i - \sum_{j \in N} A_{ij}x_j + x_0 \quad (i \in B),$$

và mục tiêu là cực đại hóa $-x_0$. Nghĩa là trong bài phụ, N có thêm chỉ số 0, B và b giữ nguyên, $A_{i0} = -1$ với mọi $i \in B$, các phần tử khác của A giữ nguyên; $c_0 = -1$ còn các c_i khác bằng 0.

Vì x_0 cũng bị ràng buộc không âm, giá trị tối đa của bài phụ không thể vượt 0. Nếu giá trị tối đa bằng 0, tức $x_0 = 0$, thì điều kiện của bài phụ trở lại đúng điều kiện ban đầu, và dạng lỏng thu được có mọi $b_i \geq 0$. Nếu giá trị tối đa nhỏ hơn 0, điều kiện không thể đồng thời thỏa, nên bài toán gốc vô nghiệm.

Vẫn còn vấn đề là ở bài phụ, ban đầu b có thể âm. Nhưng ta có cách đơn giản để sửa: chọn b_l nhỏ nhất rồi thực hiện một lần $\text{pivot}(l, \theta)$; sau đó b sẽ không còn phần tử âm.

Tiếp theo gọi opt để giải bài phụ. Nếu tối ưu của bài phụ nhỏ hơn 0, trả về vô nghiệm. Nếu tối ưu bằng 0, lấy N, B, A, b hiện tại làm dạng lỏng cho bài gốc. Ta còn phải loại x_0 . Nếu lúc này $0 \in B$, chọn tùy ý một $e \in N$ rồi thực hiện $\text{pivot}(\theta, e)$ để đưa x_0 trở thành biến phi cơ sở. Vì $x_0 = b_0 = 0$, pivot này không ảnh hưởng đến b theo nghĩa cần thiết. Sau đó loại 0 khỏi N và bỏ mọi chỉ số liên quan đến 0 trong A .

Việc cuối cùng là phục hồi vector c của hàm mục tiêu gốc. Với mỗi biến x_i trong mục tiêu gốc mà hiện thuộc B , ta thay

$$x_i = b_i - \sum_{j \in N} A_{ij}x_j$$

vào hàm mục tiêu. Sau đó mục tiêu lại tương đương với bài gốc, và khởi tạo hoàn tất.

Mã giả:

```
init
    find l with minimum b[l]
    if b[l] >= 0
        return

    origC = c
    add 0 to N
    for i in B
        A[i][0] = -1
    for i in N
        c[i] = 0
    c[0] = -1
    // auxiliary problem constructed

    pivot(l, 0)
    opt
    if v < 0
        return infeasible

    remove 0, handling the case 0 is in B
    c = origC
    for i in B with c[i] > 0
        v = v + c[i] * b[i]
        for j in N
            c[j] = c[j] - A[i][j] * c[i]
        c[i] = 0
    return initialized
```

3.5 Chi tiết và tối ưu

Thuật toán đơn hình dùng số thực, nên sai số là không thể tránh. Trong cài đặt, các so sánh với 0 nên được thay bằng một hằng rất nhỏ ϵ để tránh những kết quả vô lý do sai số tích lũy.

Điều ta quan tâm nhất là tốc độ. Thao tác chính của đơn hình là pivot; số pivot càng ít thì chương trình càng nhanh. Một cách đơn giản để giảm số pivot là tối ưu cách chọn e và l trong opt. Chẳng hạn, chọn cặp (e, l) làm v tăng nhiều nhất; nếu mức tăng bằng nhau thì dùng quy tắc Bland. Cần chú ý rằng giá trị tăng tốt nhất ban đầu không nên đặt là 0, vì có những pivot có mức tăng 0 nhưng vẫn cần thực hiện, không thể nhầm với trạng thái tối ưu. Thực nghiệm ở mục sau cho thấy tối ưu tham lam đơn giản này giúp tốc độ tăng đáng kể.

Bài tập UVA 10498

Bài toán có nội dung như sau. Một người muốn mua một gói thức ăn cho m người, và mọi người đều ăn gói giống nhau. Gói gồm n loại thực phẩm, biết đơn giá của từng loại. Do khẩu vị khác nhau, mỗi người nhận được một đơn vị một loại thực phẩm sẽ có mức thỏa mãn khác nhau; các giá trị này đã biết. Mỗi người có một giới hạn thỏa mãn đã biết. Hãy tìm số tiền lớn nhất có thể chi mà không vượt giới hạn thỏa mãn của bất kỳ ai. Lượng mỗi loại thực phẩm có thể là số thực.

Input gồm n, m , sau đó là n số biểu diễn đơn giá, rồi m dòng, mỗi dòng có $n + 1$ số. Trong dòng thứ i , n số đầu là mức thỏa mãn của người thứ i khi nhận một đơn vị từng loại thực phẩm, số cuối là giới hạn thỏa mãn. Output là một dòng chứa giá cao nhất, làm tròn lên.

Đây là một bài quy hoạch tuyến tính rất trực tiếp, dữ liệu đã ở dạng chuẩn. Hơn nữa b ban đầu toàn không âm, nên không cần viết thủ tục khởi tạo. Toàn bộ chương trình chỉ hơn 100 dòng; bản gốc nhắc đến mã nguồn trong phụ lục.

3.6 Thử nghiệm

Tác giả so sánh chương trình của mình với LINDO và MATLAB 7.0. LINDO cũng dùng đơn hình, còn MATLAB 7.0 dùng phương pháp điểm trong. Trước khi xem kết quả, nhắc ngắn gọn hai thuật toán khác cho quy hoạch tuyến tính:

Phương pháp ellipsoid. Về mặt lý thuyết là thuật toán đa thức, nhưng hiệu quả thực tế kém đơn hình. Nhiều thực nghiệm lớn cho thấy ellipsoid thua đơn hình; thuật toán đơn hình tuy có lý thuyết xấu hơn, nhưng thực tế thường tốt hơn nhiều.

Phương pháp điểm trong. Cũng có bảo đảm đa thức và được cho là có hiệu quả thực tế tốt. Tuy nhiên tác giả không có so sánh thật sự công bằng với đơn hình: MATLAB dùng phương pháp này, nhưng khi thử dữ liệu lớn, việc đọc dữ liệu khiến máy chạy rất chậm.

Dữ liệu thử nghiệm được sinh ngẫu nhiên bằng chương trình `gen.cpp` trong phụ lục gốc. Dữ liệu không nhất thiết rất mạnh, nhưng đủ để phản ánh tốc độ chương trình. Vì chỉ nhằm minh họa đại khái, sai số thời gian có thể khoảng 1 giây.

Dữ liệu nhỏ

Chương trình của tác giả ở bảng này là bản chưa thêm tối ưu tham lam. MATLAB đã chậm ở quy mô (100, 100), nên không được thử với dữ liệu lớn hơn.

m	n	Kết quả
50	50	Chương trình: tức thì; MATLAB 7.0: tức thì; LINDO: tức thì
50	100	Chương trình: tức thì; MATLAB 7.0: 17 giây; LINDO: tức thì
100	50	Chương trình: tức thì; MATLAB 7.0: 17 giây; LINDO: tức thì
100	100	Chương trình: rất nhanh; MATLAB 7.0: 30 giây; LINDO: rất nhanh

Dữ liệu lớn hơn

m	n	Kết quả
200	200	Bản thường: 8 giây, 5396 pivot; bản tham lam: 1 giây, 336 pivot; LINDO: 1 giây, 558 lần lặp
200	300	Bản thường: 11 giây, 5310 pivot; bản tham lam: 1 giây, 349 pivot; LINDO: 1 giây, 578 lần lặp
300	200	Bản thường: 28 giây, 7864 pivot; bản tham lam: 2 giây, 485 pivot; LINDO: 2 giây, 819 lần lặp
300	300	Bản thường: 105 giây, 18243 pivot; bản tham lam: 7 giây, 947 pivot; LINDO: 7 giây, 1369 lần lặp

Tỉ lệ thời gian giữa bản thường và bản tham lam không bằng tỉ lệ số pivot, vì trước mỗi pivot, bản tham lam làm thêm việc chọn cặp (l, e) giúp mức tăng lớn nhất.

Dữ liệu giới hạn

m	n	Kết quả
400	400	Bản tham lam: 23 giây, 1368 pivot; LINDO: 17 giây, 2487 lần lặp
400	500	Bản tham lam: 22 giây, 1078 pivot; LINDO: 11 giây, 1770 lần lặp
500	400	Bản tham lam: 36 giây, 1521 pivot; LINDO: 24 giây, 2686 lần lặp
500	500	Bản tham lam: 53 giây, 1837 pivot; LINDO: 59 giây, 4022 lần lặp

Các bảng cho thấy bản tham lam thường có số thao tác ít hơn, nhưng tổng thời gian nhiều lúc vẫn không tốt hơn LINDO. Tác giả cho rằng nguyên nhân có thể là LINDO tối ưu rất mạnh bản thân thao tác pivot, có lẽ liên quan đến cách dùng ma trận nghịch đảo. Phần này được để người đọc tự tìm hiểu thêm.

Tổng kết

Quy hoạch tuyến tính thường dùng để tìm một nghiệm khả thi thỏa các ràng buộc tuyến tính, hoặc tìm giá trị tối ưu của một hàm mục tiêu tuyến tính. Các mô hình như phân bổ tài nguyên, cung ứng vật tư tối ưu và luồng đa hàng hóa đều có thể được diễn đạt bằng quy hoạch tuyến tính. Nó có thể được giải bằng thuật toán đơn hình. Dù đơn hình không có độ phức tạp đa thức trong lý thuyết, hiệu quả thực tế rất tốt; với 100 biến và 100 ràng buộc, kết quả gần như xuất hiện tức thì. Việc chọn l, e theo tham lam trong đơn hình thường đem lại cải thiện tốc độ rõ rệt. Trong nhiều bài, khởi tạo là không cần thiết, nên chương trình rất ngắn và dễ viết.

Ứng dụng của quy hoạch tuyến tính trong thi tin học hiện vẫn ít, nhưng trong thực tế nó đã được dùng rất rộng rãi. Bài viết này chỉ giới thiệu sơ lược, vì thiếu các ví dụ thi đấu kinh điển nên tính thực dụng và kỹ thuật trong thi chưa thể sánh với nhiều chủ đề khác. Hy vọng người đọc tiếp tục suy nghĩ để quy hoạch tuyến tính một ngày nào đó có thể được dùng trong thi tin học rộng rãi như luồng mạng.

Lời cảm ơn

Bản gốc ghi rằng phần cảm ơn vẫn đang được bổ sung.

Tài liệu tham khảo

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*.
- Zhang Huien, *Management Linear Programming*.

Phụ lục trong bản gốc

Phụ lục của PDF chỉ liệt kê tên các tệp đi kèm, không trích được nội dung mã nguồn trong phần văn bản: `Lp.cpp` cho chương trình đơn hình, `LP_greedy.cpp` cho bản có tối ưu tham lam, `10498.cpp` cho UVA 10498, `Gen.cpp` cho sinh dữ liệu, hai bản `Translator.cpp` để chuyển input sang lệnh MATLAB hoặc tệp LINDO, và `ToStandard.cpp` để chuyển dạng tổng quát sang dạng chuẩn. Bản gốc nói các tệp này nằm trong thư mục `LinearProgramming` cùng một `Readme.txt`. Đề tiếng Anh của UVA 10498 cũng chỉ được ghi chú là tạm thời không lấy được.